

AI Task — IoT Anomaly Detection Service

Role: AI Engineer (Python) · **Est. effort:** ~3–4 hours · **Deliverable:** Git repo (link or zip)

Goal

Build a small service that runs an AI model on **IoT sensor data**, exposes it via a **REST API**, and is **containerized with Docker**. We're looking for clean, sensible engineering — not a production system. If you run out of time, document what you'd do next.

Use case: Anomaly detection / predictive maintenance

Given a window of recent sensor readings from a machine (e.g. temperature, rotational speed, torque, tool wear), **flag whether it looks abnormal** — an early sign of possible failure. Output an anomaly score and a boolean flag.

What to build

1. **Model** — Load or train a compact anomaly-detection model on the sensor data (fit on "normal" data, score new windows). **No GPU required**
2. **REST API** (FastAPI or Flask):
 - `POST /predict` — accepts a JSON payload of sensor readings, returns `{ "anomaly_score": ..., "is_anomaly": true/false }`.
 - `GET /health` — health check
3. **Docker** — a `Dockerfile` so we can build and run with two commands. Load the model once at startup, not per request
4. **README** — model choice & why; how you preprocessed the data (windowing, normalization, handling missing/irregular readings); exact build/run commands; an example `curl` request + response; trade-offs and next steps (what you'd change for real device streams on **Azure**).

Example dataset

AI4I 2020 Predictive Maintenance Dataset (UCI, synthetic industrial telemetry, ~10k rows, labeled machine-failure flag, CC BY 4.0):

- Page: [UCI Machine Learning Repository](#)
- Or load directly: `pip install ucimlrepo` → `fetch_ucirepo(id=601)`

Note: AI4I is tabular (one row per machine snapshot), so please construct sliding windows over the rows to treat it as a sensor stream rather than solving it as plain row-wise classification.

Notes

- If anything is unclear or you have any questions, please ask — we're happy to help
- Don't over-engineer — a focused working solution beats an unfinished ambitious one